Contents

1 Glossary and Conventions **1**

2 Project **1**
- 2.1 Light . . . 1
- 2.2 Component Under Test . . . 1

3 The Transform Class **1**
- 3.1 Object Diagram . . . 1
- 3.2 Currently Known Flaw . . . 3

4 Quality Criteria **3**

5 transform.cpp **3**
- 5.1 Transform::AddChild(Transform* tr) . . . 3
- 5.2 Transform::AddComponent(Component* component) . . . 4
- 5.3 Transform::FindChildByName(std::string name) . . . 4
- 5.4 Transform::FindComponentByUID(uint64$_t$ id) . . . 5
- 5.5 Transform::FindTransformByUID(uint64$_t$ id) . . . 5
- 5.6 Transform::GetComponent(std::string name) . . . 5
- 5.7 Transform::GetGlobalPosition() . . . 5
- 5.8 Transform::GetGlobalRotation() . . . 5
- 5.9 Transform::GetGlobalScale() . . . 5
- 5.10 Transform::Rotate(glm::vec3 axis, float amount) . . . 6
- 5.11 Transform::Scale(glm::vec3 amount) . . . 6
- 5.12 Transform::Translate(glm::vec3 offset) . . . 6

1 Glossary and Conventions

Term	Definition

2 Project

2.1 Light

Light is a minimalist game engine, designed to allow a modern, editor-based workflow while being lightweight enough to run on the vast majority of modern hardware. It was written over a period of about 20 days for CSC455 at SUNY Oswego.

2.2 Component Under Test

The focus of the testing will be limited to the ‘Transform‘ class due to time constraints. This class is used to build a tree structure representing a 3D scene, along with allowing pieces of C++ code (‘Component‘s) to be attached to nodes in this tree. The vast majority of the other features of the engine rely on this class, so it is the most critical to test. Additionally, it is known to have at least one flaw.

3 The Transform Class

3.1 Object Diagram

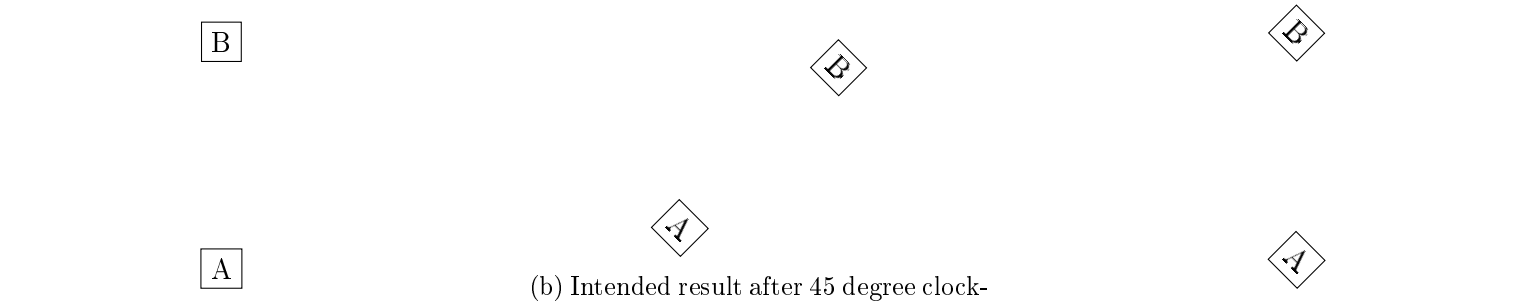
[breaklines=true,breakanywhere=true,fontsize=]plantuml @startuml allowmixing
object "t1 : Transform" name = "t1" id = 1 position = <U+0028>0,0,0<U+0029> rotation = <U+0028>0,0,0,1<U+0029> scale = <U+0028>1,1,1<U+0029>
object t2
t2 : name = "t2" t2 : id = 2 t2 : position = <U+0028>1,0,0<U+0029> t2 : rotation = <U+0028>0,0,0,1<U+0029> t2 : scale = <U+0028>1,1,1<U+0029>
object t3

```
    t3 : name = "t3" t3 : id = 3 t3 : position = <U+0028>1,0,1<U+0029> t3 : rotation = <U+0028>0,0,0,1<U+0029> t3 : scale
= <U+0028>1,1,1<U+0029>
    object "c1 : Component" "c1 : Component" : name = "c1" "c1 : Component" : id = 4
    "t1 : Transform" <- t2 "t1 : Transform" <- t3
    t3 o-right- "c1 : TestComponent"
    object t4 name = "t4" id = 5 position = <U+0028>1,0,1<U+0029> rotation = <U+0028>0,0,0,1<U+0029> scale = <U+0028>1,1,1

    t3 <- t4
    @enduml
```

3.2 Currently Known Flaw

To illustrate the flaw, imagine a transform called "A" at the origin, with a transform "B" as its child, translated along the Y axis an arbitrary distance. If the transform "A" is rotated about its Z axis, the child should follow the parent, such that it maintains its position and rotation relative to the parent. Instead, the child rotates independently.



(a) Initial setup, A is the parent of B

(b) Intended result after 45 degree clockwise rotation of A

(c) Actual result with current code

4 Quality Criteria

Quality Property	Definition	Metric	Criterion
Functional Correctness	Test cases yield the expected results anticipated during derivation.	Number of test cases that pass.	90% of test cases pass.
Fault Tolerance	Software should run as intended even if invalid inputs are supplied.	Number of test cases that produce an unrecoverable error, such as a segfault.	All test cases properly recover from invalid inputs.
Performance	Software should run efficiently enough to allow smooth rendering.	Time taken to update state and render a frame.	Average frame time is under 16ms (60 frames per second or higher)

5 transform.cpp

5.1 Transform::AddChild(Transform* tr)

5.1.1 Specification

[breaklines=true,breakanywhere=true,fontsize=]c++ /** * Add a child transform to this transform * * If the desired child is already a child of this transform * log a warning and ignore it * * @param tr the transform to add as a child * */ void AddChild(Transform *tr);

5.1.2 Equivalence Classes

Eq. Class ID	Param	Description	Representative
1.1.EC1.1	tr	a valid pointer	new Transform()
1.1.EC1.a	tr	a null pointer	nullptr

5.1.3 Test Cases

Test Case ID	Param. *tr*	Expected
1.1.TC1	A pointer to some valid Transform	No output
1.1.TC2	nullptr	Error: log message

5.1.4 Test Implementation

[breaklines=true,breakanywhere=true,fontsize=]c++ TEST$_CASE$("1.1.TC1")
Transform* test$_transform$ = $newTransform()$; $Transform * child_transform$ = $newTransform()$;
//redirect cout to a string so we can check for the error message std::stringstream buffer; std::streambuf* old$_cout$ = std :: $cout.rdbuf(buffer.rdbuf())$;

test$_transform->AddChild(child_transform)$;
std::cout.rdbuf(old$_cout$);$CHECK(buffer.str() == "")$;//nooutputexpected
delete child$_transform$;$deletetest_transform$;
TEST$_CASE$("1.1.TC2")
Transform* test$_transform = newTransform()$;
std::stringstream buffer; std::streambuf* old$_cout = std :: cout.rdbuf(buffer.rdbuf())$;
test$_transform->AddChild(nullptr)$;
std::cout.rdbuf(old$_cout$);$CHECK(buffer.str() != "")$;//somekindoflogoutputexpected
delete test$_transform$;

5.2 Transform::AddComponent(Component* component)

5.2.1 Specification

[breaklines=true,breakanywhere=true,fontsize=]c++
/** * Add a component to this transform * * If the supplied pointer is null, log error and ignore * If the supplied pointer is not a component, log error and ignore * * @param component A pointer to the component to add */ void AddComponent(Component* component);

5.2.2 Equivalence Classes

Eq. Class ID	Param	Description	Representative
1.2.EC1.1	component	a valid pointer	new TestComponent()
1.2.EC1.a	component	a valid pointer to something other than a Component	new Transform()
1.2.EC1.b	component	an invalid pointer	nullptr

5.2.3 Test Cases

Test Case ID	Param. *tr*	Expected	*Covered EQ Class
1.2.TC1	new TestComponent()	No output	1.2.EC1.1
1.2.TC2	new Transform()	Error: log message and ignore	1.2.EC1.a
1.2.TC3	nullptr	Error: log message and ignore	1.2.EC1.b

5.3 Transform::FindChildByName(std::string name)

5.3.1 Equivalence Classes

Eq. Class ID	Param	Description	Representative
1.3.EC1.1	name	A string	"string"
1.3.EC1.a?	name	The empty string	""
1.3.EC1.b	name	Not a string	1

5.3.2 Test Cases

Test Case ID	Param. *name*	Expected
1.1.TC1.1	"string"	nullptr
1.1.TC1.2	""	nullptr
1.1.TC1.b	1	This will be disallowed by type checking, so TC implementation is omitted.

- The specification is unclear on whether the empty string is a valid name for a Transform, so I am assuming it is a valid transform name.

5.4 Transform::FindComponentByUID(uint64$_t$ id)

5.4.1 Equivalence Classes

Eq. Class ID	Param	Description	Representative
1.4.EC1.1	id	A positive integer	1
1.4.EC1.2	id	Zero	0
1.4.EC1.a	id	A negative integer	-1

5.4.2 Test Cases

Test Case ID	Param. *tr*	Expected	Coverage
1.4.TC1.1	1	nullptr	1.4.EC1.1
1.4.TC1.2	0	nullptr	1.4.EC1.2
1.4.TC1.a	-1	Disallowed by type checking, TC implementation omitted	1.4.EC1.a

- 1.1.TC1.a omitted, as it would be disallowed by the type checker.

5.5 Transform::FindTransformByUID(uint64$_t$ id)

5.5.1 Equivalence Classes

Equivalence classes same as Transform::FindComponentByUID.

5.5.2 Test Cases

Test Case ID	Param. *tr*	Expected	Coverage
1.5.TC1.1	1	nullptr	1.5.EC1.1
1.5.TC1.2	0	nullptr	1.5.EC1.2
1.5.TC1.a	-1	Disallowed by type checking, TC implementation omitted	1.5.EC1.a

5.6 Transform::GetComponent(std::string name)

5.6.1 Equivalence Classes

Possible issues with specification:

- The method to get a transform by name is called ‘FindChildByName‘, this method should be updated to match that scheme as ‘FindComponentByName‘.

  The equivalence classes are the same as Transform::FindChildByName.

5.7 Transform::GetGlobalPosition()

5.7.1 Equivalence Classes

5.8 Transform::GetGlobalRotation()

5.8.1 Equivalence Classes

Equivalence classes same as Transform::GetGlobalPosition.

5.9 Transform::GetGlobalScale()

5.9.1 Equivalence Classes

Equivalence classes same as Transform::GetGlobalScale.

5.10 Transform::Rotate(glm::vec3 axis, float amount)

5.10.1 Equivalence Classes

Eq. Class ID	Param	Description	Representative
1.14.EC1.1	axis	axis is a vector with nonzero length	{1.0,0.0,0.0}
1.14.EC1.a	axis	axis is a vector with zero length	{0.0,0.0,0.0}
1.14.EC1.b	axis	axis is not a vector	1
1.14.EC2.1	amount	amount is a float	90.0f
1.14.EC2.2	amount	amount is an integer	90
1.14.EC2.a	amount	amount is not a number	"90"

5.10.2 Test Cases

Test Case ID	Param. *axis*	*amount*	Expected	Additional Coverage
1.14.TC1	{1.0,0.0,0.0}	90.0f	No output	1.14.EC1.1 1.14.EC2.1
1.14.TC2	{1.0,0.0,0.0}	90	No output	1.14.EC2.2
1.14.TC3	{0.0,0.0,0.0}	90.0f	Error: log message	1.14.EC1.a
1.14.TC4*	1	90.0f	Disallowed by type checker	1.14.EC1.b
1.14.TC5*	{1.0,0.0,0.0}	"90"	Disallowed by type checker	1.14.EC2.a

5.11 Transform::Scale(glm::vec3 amount)

5.11.1 Equivalence Classes

Eq. Class ID	Param	Description	Representative
1.15.EC1.1	amount	Any vector other than the zero vector or the identity vector of component-wise multiplication	{x=2.0,y=3.0,z=2.0}
1.15.EC1.2	amount	The identity element of component-wise multiplication	{x=1.0,y=1.0,z=1.0}
1.15.EC1.3	amount	The zero vector	{x=0.0,y=0.0,z=0.0}
1.15.EC1.a	amount	Not a vector	1

5.11.2 Test Cases

Test Case ID	Param. *amount*	Expected	Coverage
1.15.TC1	{2.0,3.0,2.0}	No output	1.15.EC1.1
1.15.TC2	{1.0,1.0,1.0}	No output	1.15.EC1.2
1.15.TC3	{0.0,0.0,0.0}	No output	1.15.EC1.3
1.15.TC4*	1	Disallowed by type checker	1.15.EC1.a

5.12 Transform::Translate(glm::vec3 offset)

5.12.1 Equivalence Classes

Eq. Class ID	Param	Description	Representative
1.16.EC1.1	amount	Any vector that is not the identity vector	{x=2.0,y=3.0,z=2.0}
1.16.EC1.2	amount	The identity vector for addition	{x=0.0,y=0.0,z=0.0}
1.16.EC1.a	amount	Not a vector	1

5.12.2 Test Cases

Test Case ID	**Param.** *amount*	**Expected**	Coverage
1.16.TC1	{2.0,3.0,2.0}	No output	1.16.EC1.1
1.16.TC2	{0.0,0.0,0.0}	No output	1.16.EC1.2
1.16.TC3	1	Disallowed by type checker	1.16.EC1.a